Backtesting Engine Software Design Document

1. Introduction

1.1 Purpose

This Software Design Document (SDD) outlines the architecture and components of the Backtesting Engine system. It is intended to guide developers through the implementation of the system and ensure that design decisions align with the requirements defined in the SRS.

1.2 Scope

This document covers the software design of the standalone Python application that will simulate the execution of algorithmic trading strategies on historical financial data. It includes component definitions, module descriptions, data flow, interface designs, and implementation strategies.

1.3 Definitions, Acronyms, and Abbreviations

- Backtesting Engine: A software tool used to evaluate the effectiveness of trading strategies using historical market data
- Positions: The quantity of a security held in a trading account at any point in time (long, short, or neutral).
- Equity Curves: Graphs that depict the value of a trading account over time, used to visualize strategy performance.
- Trade Markers: Visual indicators on charts that denote where buy and sell actions were executed.
- SRS: Software Requirement Specification
- CSV: Comma-Separated Values
- PnL: Profit and Loss
- MA: Moving Average
- API: Application Programming Interface
- CLI: Command Line Interface

1.4 Intended Audience and Reading Suggestions

This document is intended for developers and testers who are interested in quantitative finance and software development. It is structured to define functional and non-functional requirements, system features, and high-level design assumptions

2. System Overview

The system is composed of several modules:

- Data Loader: Loads historical financial data (CSV)
- Strategy Interface: Provides a list of predefined strategy modules (which are implemented as Python classes). Each strategy must implement a generate_signal() method and may also implement prepare()
- Simulation Engine: Steps through time, applies strategy logic, and simulates trades
- Portfolio Manager: Tracks cash, holdings, PnL, and metrics
- Visualization Module: Generates equity curves and trade markers
- Exporter: Outputs performance reports and logs

These modules will interact within a modular Python application using shared data structures and internal interfaces.

3. Design Considerations

3.1 Assumptions and Dependencies

- Python 3.9+
- External libraries: pandas, numpy, matplotlib
- Local-only execution with pre-downloaded datasets

3.2 General Constraints

- Modular structure for maintainability
- No reliance on external APIs during runtime

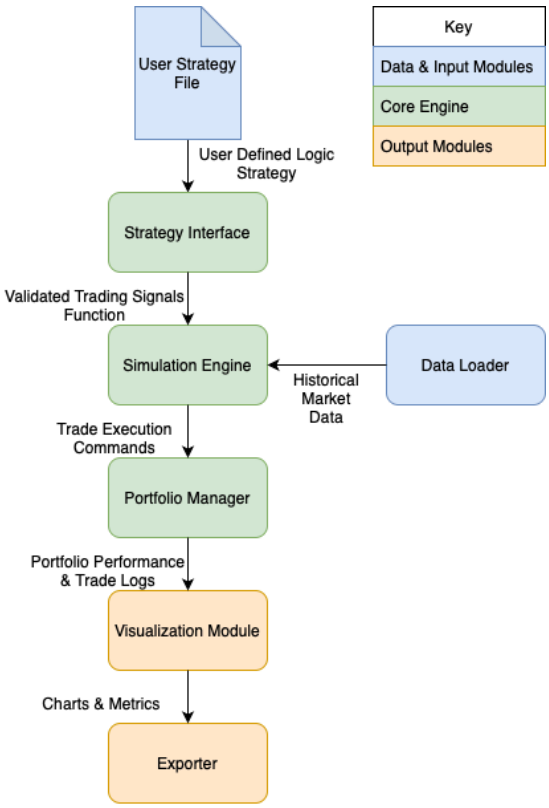
3.3 Goals and Guidelines

- Prioritize simplicity and readability
- Minimize runtime errors through input validation

4. Architectural Design

4.1 High-Level Architecture

A modular architecture will be used:



4.2 Component Descriptions

- Data Loader: Reads historical data and formats for simulation.
- Strategy Interface: Loads and manages predefined strategy modules that implement generate_signal() method.
- Simulation Engine: Controls the time iteration loop and applies the strategy logic to each timestep.
- Portfolio Manager: Calculates position size, updates cash balances, and logs trades.
- Visualization Module: Uses matplotlib to generate graphs (equity curves, price with markers).
- Exporter: Saves results and metrics to CSV or .txt files.

5. Data Design

5.1 Data Structures

- MarketData: pandas DataFrame (Date, Open, High, Low, Close, Volume)
- PortfolioState: custom class tracking cash, holdings, PnL, and trades.
- TradeLog: List of dictionaries or pandas DataFrame recording tables

5.2 Data Flow

1. Load Data -> 2. Run Strategy -> 3. Update Portfolio -> 4. Visualize & Export Results

6. Interface Design

6.1 User Interface

Initial version will use a command line interface (CLI) with prompts for:

- Selection of strategy and ticket from the predefined options via command line prompts
- Outputs are auto-saved into the results/ folder

6.2 Software Interfaces

- Use pandas for data handling
- Use matplotlib for plotting

7. Appendix

Sample strategy format

```
class SampleStrategy:
    def prepare(self, df):
        # Optional: compute indicators
        pass

    def generate_signal(self, index, row, df):
        # Return one of: "BUY", "SELL", or "HOLD"
        return "HOLD"
```

Folder Structure

```
backtesting-engine/
├── core/
│   ├── backtester.py
│   ├── data_loader.py
│   └── portfolio.py
│
├── strategies/
│   ├── sample_strategy.py
│   ├── rsi_strategy.py
│   ├── momentum_strategy.py
│   ├── bollinger_band_strategy.py
│   └── frequent_trading_strategy.py
│
├── utils/
│   ├── metrics.py
│   └── plotter.py
│
├── data/
│   └── AAPL_data.csv
│
├── results/
│   ├── summary.csv
│   └── [ticker]/[strategy]/...
│
├── main.py
├── requirements.txt
└── README.md
```

High-Level Architecture Diagram